PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS *CAMPUS* POÇOS DE CALDAS
CURSO CIÊNCIAS DA COMPUTAÇÃO

BRENDA CAROLINE TRISTÃO
DANIELY FERNANDA POSCIDONIO
JOÃO GABRIEL BERNARDES DE OLIVEIRA
GABRIEL ALENCAR DO LAGO
YASMIN KAROLINE ALCISIO CORONA
HENRIQUE DE SOUZA ROCHA

SALVOU: Software de Gestão de Clientes para Autônomos e Microempreendedores Individuais (MEIs)

POÇOS DE CALDAS - MG

2026

1. **Funcionalidade Principal:** Cadastro de Clientes

2. **Descrição do Fluxo Completo (João Gabriel)**

- **Quem inicia a ação:**

O fluxo é iniciado pelo usuário (profissional autônomo) ao acessar o sistema e escolher a opção de cadastrar um novo cliente.

- **O que acontece primeiro**

Após realizar login no sistema, o usuário:

- Acessa a tela de cadastro de clientes
- Preenche os dados básicos, como:
    - Nome;
    - Telefone;
    - E-mail;
    - CPF;
    - Data de nascimento;
    - Endereço.

Essas informações são essenciais para criar o perfil do cliente.

- **Quais partes do sistema são acionadas**

  Durante o fluxo, diferentes partes do sistema são acionadas:

    - Interface (Frontend): exibe o formulário e coleta os dados
    - Camada de controle: recebe os dados inseridos
    - Camada de serviço: valida e processa as informações
    - Camada de persistência: salva os dados no banco

- **Como os dados circulam**

  O fluxo de dados ocorre da seguinte forma:

    - O usuário insere os dados no formulário
    - O sistema envia esses dados para o Controller
    - O Controller encaminha para o Service
    - O Service:
    - Valida os dados (ex: campos obrigatórios)
    - Aplica regras de negócio (ex: evitar cliente duplicado)
    - Após validação, os dados são enviados para o Repository
    - O Repository realiza a gravação no banco de dados
    - O sistema retorna uma resposta de sucesso para o usuário

- **Qual é o resultado final**

Ao final do fluxo:

- O cliente é registrado no sistema com sucesso
- Seus dados ficam armazenados e organizados
- O profissional pode consultar e atualizar essas informações a qualquer momento
- O sistema passa a utilizar esses dados em outras funcionalidades, como:
  - Agendamentos
  - Relatórios
  - Histórico de relacionamento

3. Mapeamento Técnico (Brenda)

O fluxo de cadastro transforma uma intenção do usuário em um dado persistente, atravessando as camadas do sistema para garantir que a informação seja válida, segura e organizada.

1. Identificação das Classes e Papéis

O sistema utiliza uma arquitetura em camadas (Controller-Service-Repository), onde cada componente possui uma responsabilidade única:

Interface do Usuário (ClienteView - React):

Papel: Componente de Frontend.

Responsabilidade: Apresentar o formulário, capturar os dados digitados pela Joyce (Nome, CPF, Telefone, etc.) e disparar uma requisição HTTP (JSON) para o backend.

Camada de Controle (ClienteController):

Papel: "Porteiro" da API.

Responsabilidade: Receber a requisição, validar se os dados chegaram no formato correto (sintaxe) e encaminhar a execução para a camada de serviço.

Camada de Negócio (ClienteService):

Papel: "Cérebro" do sistema.

Responsabilidade: Implementar as regras de negócio. Verifica se o CPF já existe no banco, valida regras de integridade e decide se o cliente pode ou não ser salvo.

Modelo de Dados (Classe Cliente):

Papel: Entidade/Objeto de Negócio.

Responsabilidade: Representar o cliente dentro do código, contendo os atributos (ID, nome, e-mail, status) e métodos básicos.

Camada de Persistência (ClienteRepository):

Papel: Especialista em Banco de Dados (Spring Data JPA).

Responsabilidade: Lidar exclusivamente com a comunicação com o banco SQL, transformando o objeto Java em comandos de gravação ou consulta.

2. Comunicação entre as Classes (O Caminho do Dado)

- O fluxo de dados ocorre de forma linear e descendente, garantindo o desacoplamento do sistema:
- Entrada: A Joyce preenche os dados na ClienteView e clica em "Salvar".
- Tráfego: A Interface envia um objeto JSON para o ClienteController.
- Encaminhamento: O Controller recebe os dados e aciona o método correspondente no ClienteService.
- Processamento: O Service aplica a lógica de negócio (ex: evitar duplicidade de CPF).
- Gravação: Após a validação, o Service solicita ao ClienteRepository que persista os dados.
- Confirmação: O Repository grava no banco de dados e retorna a confirmação para o Service, que repassa ao Controller, resultando na mensagem de sucesso exibida na Interface.

3. Análise Crítica: Arquitetura vs. Design

Arquitetura (Macro): A separação em camadas (Controller -> Service -> Repository) garante que o sistema seja fácil de manter. Se o banco de dados mudar, apenas o Repository é afetado. Se uma regra de negócio mudar, apenas o Service é alterado.

Design (Micro): O detalhamento da classe Cliente (definindo tipos para CPF e Telefone) e a sua relação com classes como HistoricoCliente e Observacao (conforme os Cartões CRC) permitem que o sistema cresça sem perder a organização dos dados da Joyce.

4. **Organização Arquitetural (Gabriel Alencar e Henrique)**

Fluxo Funcionalidade 1:
Cadastro e gerenciamento de clientes

Estrutura de Sistema

1. Interface do Usuário (Frontend - React):
    - Onde está: No navegador do usuário.
    - Responsabilidade: Apresentar o formulário com os campos (Nome, Telefone, E-mail, CPF, etc.) e capturar a entrada da Joyce.
    - Comunicação: Dispara uma requisição HTTP POST para o backend.

2. Camada de Controle (Controller):
    - Classe: ClienteController
    - Responsabilidade: Atuar como o "garçom" do sistema. Ele recebe a requisição, valida se o JSON enviado está no formato correto (ex: se o CPF não está em branco) e repassa a tarefa para o serviço.
    - Comunicação: Recebe do Frontend - Aciona o ClienteService.

3. Camada de Negócio (Service):
    - Classe: ClienteService
    - Responsabilidade: Implementar as regras de negócio. Antes de salvar, esta camada verifica, por exemplo, se o CPF informado já existe no banco ou se o endereço está completo. É o "cérebro" que garante a integridade dos dados antes da persistência.
    - Comunicação: Recebe do Controller - Executa a lógica - Aciona o ClienteRepository.

4. Camada de Persistência (Repository):
    - Classe: ClienteRepository (Interface baseada em Spring Data JPA)
    - Responsabilidade: Lidar exclusivamente com o banco de dados. Ela transforma os objetos Java em comandos SQL para salvar ou buscar informações no Banco de Dados.
    - Comunicação: Recebe do Service - Salva no Banco de Dados.

Fluxo Simplificado da Funcionalidade

Joyce *usuário* (Interface) - ClienteController - ClienteService - ClienteRepository - Banco de Dados (SQL)

Análise Crítica das Responsabilidades

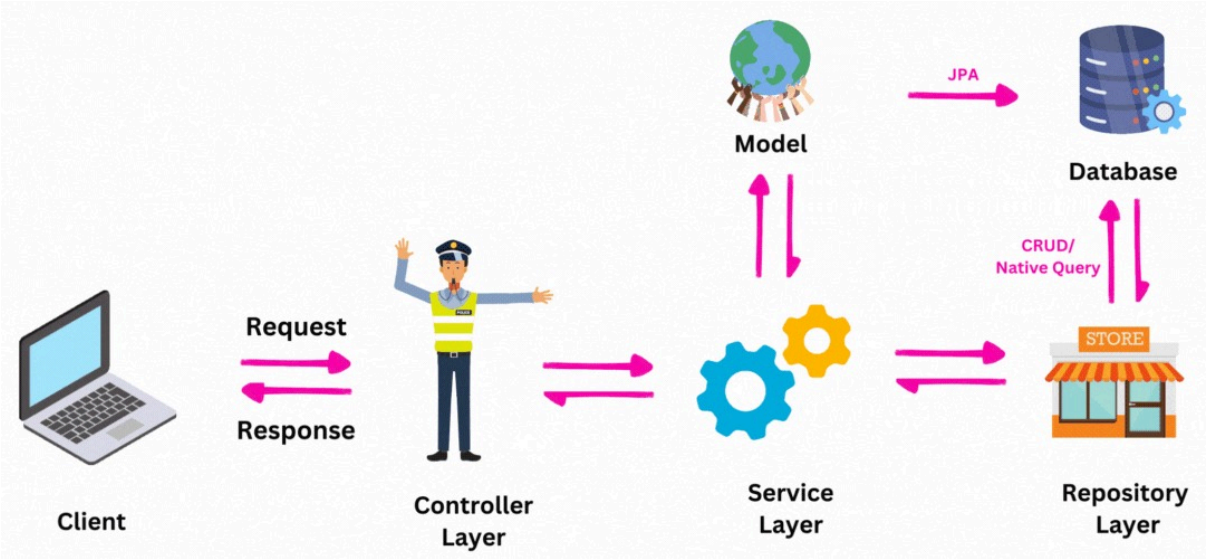
O Controller chama o Service, mas não sabe como a regra de negócio funciona internamente.

O Service chama o Repository, mas não sabe se os dados estão sendo salvos em um banco MySQL, PostgreSQL ou na nuvem.

Diferenciação entre Arquitetura (Macro) e Design (Micro)

Arquitetura (Macro): É a decisão de usar Spring Boot com camadas (Controller -> Service -> Repository). É o "esqueleto" que garante que os dados da Joyce circulem com segurança.

Design (Micro): É como você detalha a classe Cliente, definindo os tipos de dados para CPF, Telefone e Endereço, e como os métodos internos validam essas informações.



5. Identificação de Problemas (Yasmin)

1. O fluxo faz sentido?

Em linhas gerais, sim. O caminho Joyce → ClienteView → ClienteController → ClienteService → ClienteRepository → Banco de Dados é lógico, linear e coerente com a arquitetura em camadas adotada.

No entanto, existe uma contradição importante: o fluxo arquitetural está correto, mas o modelo de classes do CRC não reflete essa separação. A classe Cliente acumula métodos como filtroClientes() e relatorio(), que deveriam pertencer ao ClienteService. Isso cria uma inconsistência entre o que o diagrama modela e o que o sistema realmente faria ao funcionar.

2. Há acoplamento indevido?

Sim. A classe Cliente foi modelada colaborando diretamente com HistoricoCliente, Observacao, FiltroClientes e Relatorio ao mesmo tempo. Esse acoplamento é excessivo para uma entidade de domínio: qualquer alteração nessas classes exige revisão em Cliente, tornando o sistema frágil.

O correto é que a classe Cliente conheça apenas suas entidades filhas diretas (HistoricoCliente e Observacao), e que filtros e geração de relatórios sejam responsabilidade exclusiva do ClienteService ou de serviços auxiliares dedicados (MARTIN, 2003).

3. Alguma classe possui responsabilidades excessivas?

Sim, o ClienteService corre esse risco. Como as classes FiltroClientes e Lembrete foram convertidas em métodos, toda essa lógica precisa ir para algum lugar — e o destino natural seria o ClienteService. Se isso não for cuidadosamente separado, esse serviço crescerá acumulando responsabilidades de cadastro e validação de clientes, filtragem e segmentação, geração de lembretes e notificações, e geração de relatórios.

A solução recomendada é criar serviços auxiliares independentes, como FiltroService e LembreteService, cada um com uma única responsabilidade. Isso respeita o Princípio da Responsabilidade Única (SRP), que estabelece que uma classe deve ter apenas um motivo para mudar (MARTIN, 2003).

4. Existe algo faltando?

4.1 Validação semântica dos dados. O fluxo prevê que o ClienteController valida o formato correto dos dados (sintaxe), mas não define onde ficam as validações de negócio: CPF válido? E-mail com formato correto? Telefone com DDD? Campos obrigatórios preenchidos? Sem isso, dados incorretos podem passar pelo Controller e ser persistidos no

banco. Essa validação deve estar explícita no ClienteService, antes de qualquer chamada ao ClienteRepository.

4.2 Tratamento de erros e respostas de falha. O fluxo descreve apenas o caminho feliz — o cadastro bem-sucedido. Não há previsão de como o sistema responde em situações de erro: CPF já cadastrado (HTTP 409 Conflict), banco indisponível (HTTP 500 Internal Server Error) ou dados ausentes (HTTP 400 Bad Request). O ClienteController deveria retornar respostas HTTP adequadas para cada caso, e o ClienteService deveria lançar exceções específicas que o Controller captura e traduz para o cliente (LARMAN, 2007).

5. O que está bem no modelo

A escolha da arquitetura em três camadas (Controller → Service → Repository) é acertada e é o ponto mais forte do projeto. Ela garante que mudanças no banco de dados afetem apenas o Repository, que mudanças nas regras de negócio afetem apenas o Service, e que a interface possa ser alterada sem impactar a lógica de negócio.

Além disso, manter HistoricoCliente e Observacao como entidades separadas é uma decisão de design correta, pois evita que a classe Cliente cresça indefinidamente com dados de interação histórica (LARMAN, 2007). A separação entre Usuario e Autenticador também é adequada, isolando a lógica de autenticação da entidade de usuário e seguindo boas práticas de segurança.

6. **Definição de Classes (Daniely)**

As classes, responsabilidades, relacionamentos e modelagem foram definidos nos Sprints 1 e 2 que podem ser acessados no repositório do Github.